Ondrej Nebus ID:133751
Cvičenie Utorok: 16:00

Úvod

V rámci tohto zadania som mal za úlohu implementovať a porovnať rôzne algoritmy na efektívne vyhľadávanie v dynamických množinách. Tieto algoritmy zahŕňajú binárne vyhľadávacie stromy (BVS) s rôznymi prístupmi k ich vyvažovaniu, ako aj hašovacie tabuľky s rôznymi technikami riešenia kolízií. Rôzne dátové štruktúry sú vhodné pre rôzne situácie v závislosti od charakteru spracovávaných údajov, distribúcie hodnôt a vykonávaných operácií.

V rámci implementácie som sa rozhodol pre nasledujúce dátové štruktúry:

1. **Červeno-čierny strom** ako prvý prístup k implementácii binárneho vyhľadávacieho stromu (BVS), ktorý je samovyvažujúci a zabezpečuje efektívnosť operácií vkladania, mazaní a vyhľadávania v čase O(log n).
2. **Treap** ako druhý typ BVS, ktorý kombinuje vlastnosti binárneho vyhľadávacieho stromu a haldy, čím sa dosahuje vyváženie a efektívnosť operácií v priemere O(log n).
3. **Hašovacia tabuľka s dvojitým rozptylením**, ktorá umožňuje efektívne vyhľadávanie, vkladanie a mazanie položiek, pričom využíva techniku otvoreného adresovania s dynamickým prispôsobovaním veľkosti tabuľky pri prekročení preddefinovaného zaťaženia.

Testovanie a porovnávanie týchto implementácií prebiehalo na základe rôznych scenárov, pričom bol hodnotený priemerný čas potrebný na vykonanie operácií v závislosti od počtu položiek v dátovej štruktúre. Cieľom bolo určiť, ktorá z implementovaných štruktúr je najefektívnejšia v rôznych scenároch a pre rôzne typy operácií. Výsledky testov a analýza efektívnosti týchto dátových štruktúr sú podrobne zdokumentované v tejto práci, vrátane grafov a tabuliek, ktoré porovnávajú výkonnosť jednotlivých riešení.

Prehľad implementovaných dátových štruktúr

V tejto časti dokumentácie podrobne popíšem implementované dátové štruktúry, ktoré som použil na efektívne vykonávanie operácií, ako sú vkladanie, vyhľadávanie a mazanie prvkov v dynamických množinách. Mojím cieľom bolo porovnať rôzne prístupy k optimalizácii týchto operácií s ohľadom na ich časovú zložitosť a výkon v závislosti od rôznych faktorov, ako je veľkosť dátovej štruktúry a charakter vstupných údajov. Implementoval som nasledujúce dátové štruktúry:

Ondrej Nebus ID:133751
Cvičenie Utorok: 16:00

Červeno-čierny strom (Red-Black Tree)

Červeno-čierny strom je samovyvažujúci binárny vyhľadávací strom, ktorý zabezpečuje vyváženosť pomocou farieb uzlov (červená a čierna). Tento typ stromu sa riadi špecifickými pravidlami, ktoré garantujú, že najdlhšia cesta od koreňa k listu nebude dvojnásobná v porovnaní s najkratšou cestou. Vďaka týmto pravidlám červeno-čierny strom zaručuje, že operácie ako vkladanie, mazanie a vyhľadávanie môžu byť vykonávané v čase O(log n), kde n je počet prvkov v strome. Hlavnou výhodou červeno-čiernych stromov je, že tieto operácie prebiehajú za garantovaný čas bez potreby neustáleho vyvažovania, čo zjednodušuje implementáciu v porovnaní s niektorými inými samovyvažujúcimi stromami. **Vlastná implementácia funkcie delete:**
Pri implementácii funkcie **delete** v červeno-čiernom strome som sa rozhodol vyhnúť tradičnému prístupu, ktorý vyžaduje zložité opravy (tzv. fixup) po vymazaní uzla. Namiesto toho som sa zameral na jednoduchší prístup, pri ktorom uzol, ktorý je potrebné vymazať, nezmizne úplne, ale jeho charakter sa zmení na špecifickú hodnotu (napríklad na znak „deleted“). Tento prístup minimalizuje potrebu následného vyvažovania stromu a zjednodušuje celý proces mazania.

```
}
Node *deletenum( Node* root,int key) {
    if (!root) return NULL;
    if (root->key == key) {
        root->delete = 'N';
        return root;
    }
    if (key < root->key)
        return deletenum(root->left, key);
    return deletenum(root->right, key);
}
```

Táto metóda nemení štruktúru stromu ako takej, pretože uzol sa nezbavuje svojho miesta v strome, ale iba mení svoj „stav“, čím sa efektívne eliminuje jeho prítomnosť pri operáciách ako je vyhľadávanie a ďalšie. Zmena charakteru uzla zabezpečuje, že štruktúra stromu zostáva neporušená, pričom operácie sú stále efektívne a zachovávajú logaritmickú časovú zložitosť. Tento prístup je efektívny a umožňuje rýchle odstránenie uzlov bez zložitého a časovo náročného vyvažovania, ktoré by bolo potrebné v prípade tradičného algoritmu mazania červeno-čierneho stromu. Ostatne funkcie som robil podľa https://www.youtube.com/watch?v=t-oiZnplv7g&t=846s.

Ondrej Nebus ID:133751
Cvičenie Utorok: 16:00

2. Treap

Treap je dátová štruktúra, ktorá kombinuje vlastnosti binárneho vyhľadávacieho stromu a haldy. Každý uzol v treape obsahuje kľúč a prioritu, pričom kľúče sú usporiadané podľa pravidiel binárneho vyhľadávacieho stromu, zatiaľ čo priority sú usporiadané podľa pravidiel haldy – priorita rodiča je vždy menšia ako priorita jeho detí. Tento prístup umožňuje, aby operácie ako vyhľadávanie, vkladanie a mazanie prebiehali v priemere v čase O(log n), čo zabezpečuje vyváženosť dátovej štruktúry. Treap je výhodný v situáciách, kde je potrebné kombinovať efektivitu binárnych vyhľadávacích stromov s robustnosťou haldy.

3. Hašovacia tabuľka s dvojitým rozptylením (Double Hashing)

Hašovacia tabuľka je efektívna dátová štruktúra určená na rýchle vyhľadávanie, vkladanie a mazanie položiek. Pri implementácii hašovacej tabuľky som použil techniku dvojitého rozptylenia, ktorá minimalizuje pravdepodobnosť kolízií pri ukladaní kľúčov do tabuľky. Táto technika využíva dve hashovacie funkcie na určenie indexu v tabuľke, čím sa výrazne znižuje šanca, že dve položky budú kolidovať. Ak sa tabuľka zaplní viac než 70%, vykonám rehashing, čím sa zväčší veľkosť tabuľky, aby sa udržala efektivita operácií. Dvojité rozptylenie zaručuje, že operácie vkladania a vyhľadávania sa vykonávajú v priemere v čase O(1), avšak vo veľmi zriedkavých prípadoch môžu byť náročnejšie v dôsledku kolízií.

Hlavné komponenty programu

1. **Hašovacia tabuľka**:
    - Používa dvojité rozptylenie na určenie pozície kľúčov v tabuľke.
    - Dynamicky prispôsobuje veľkosť tabuľky počas vkladania, aby udržala efektivitu operácií.
2. **Rehashovanie**:
    - Pri prekročení **LOAD_FACTOR** (70%) sa vykoná rehashovanie, čím sa zväčší tabuľka a obnoví rozptýlenie kľúčov podľa novej veľkosti.
3. **Operácie**:
    - **Vkladanie (insert)**: Kľúč je vložený do tabuľky a v prípade potreby vykoná rehashovanie.
    - **Vyhľadávanie (search)**: Efektívne vyhľadávanie kľúčov pomocou dvojitého rozptylenia.
    - **Mazanie (delete)**: Kľúče môžu byť odstránené a ich pozícia v tabuľke bude označená ako "DELETED".

V každom zo súborov .c (kde sú implementované jednotlivé dátové štruktúry, ako hašovacia tabuľka alebo iné algoritmy) sa nachádza funkcia **`main`**, ktorá slúži na interaktívne testovanie a vykonávanie operácií. Táto funkcia umožňuje používateľovi zvoliť rôzne možnosti prostredníctvom jednoduchého menu a následne vykonať požadované operácie nad dátovou štruktúrou. Konkrétne, v každom súbore .c je tento **`main`** zodpovedný za nasledujúce funkcie:

Ondrej Nebus ID:133751
Cvičenie Utorok: 16:00

Hlavné operácie, ktoré zabezpečuje každý `main`:

1. **Vloženie čísla (Vlozit cislo)**: Umožňuje používateľovi vložiť kľúč do dátovej štruktúry (napr. hašovacej tabuľky).
2. **Vyhľadanie čísla (Vyhladat cislo)**: Vyhľadá zadaný kľúč v dátovej štruktúre a informuje používateľa, či bol nájdený alebo nie.
3. **Zmazanie čísla (Zmazat cislo)**: Odstráni kľúč zo štruktúry, ak existuje.
4. **Spustenie testu (Spusti test)**: Umožní používateľovi spustiť predpripravené testy na efektivitu operácií vkládania, vyhľadávania a mazania. Tieto testy sú zamerané na meranie výkonu pri práci s rôznymi veľkosťami dát.
5. **Ukončenie programu (Ukoncit program)**: Umožní používateľovi bezpečne ukončiť program.

```
1. Vlozit cislo
2. Vyhladat cislo
3. Zmazat cislo
4. Spusti test
5. Ukoncit program
Zvol moznost:
```

V rámci testovania som sa zameral na porovnanie výkonu troch rôznych dátových štruktúr: **Hašovacia tabuľka (Hash)**, **Treap strom** a **Red-Black strom**. Cieľom bolo zmerať čas vykonávania operácií ako **INSERT**, **SEARCH** a **DELETE** pre každú z týchto dátových štruktúr pri rôznych veľkostiach dát (50000, 250000, 500000 a 1000000 prvkov).

Princíp testovania

1. **Implementácia každého algoritmu**: Každá dátová štruktúra (Hash, Treap, RedBlack) bola implementovaná v samostatnom súbore. Každý z týchto súborov obsahoval funkciu **`main`**, ktorá riešila interaktívne testovanie a umožnila užívateľovi vykonávať operácie nad zvolenou dátovou štruktúrou.
2. **Funkcia na čítanie z testovacieho súboru**: V každej implementácii bola vytvorená funkcia, ktorá načítava príkazy zo súboru (napríklad `test-25.txt`). Súbor obsahoval príkazy ako `INSERT`, `SEARCH` a `DELETE`, ktoré sa následne vykonali na danej dátovej štruktúre.
3. **Testovanie operácií**: Pre každú implementáciu (Hash, Treap, RedBlack) som vykonal nasledujúce kroky:
    - **Generovanie testovacieho súboru**: Vygeneroval som textový súbor s požiadavkami na operácie **INSERT**, **SEARCH** a **DELETE**. Tento súbor mal rôzne veľkosti, pričom počet riadkov testovacieho súboru bol od 50000 až po 1000000.
    - **Spustenie testovacieho súboru**: Po načítaní testovacieho súboru sa spustili operácie na príslušnej dátovej štruktúre.
4. **Generovanie testovacieho súboru**: Testovacie súbory sa generovali s rôznymi počtami riadkov. Každý riadok v súbore obsahoval príkaz na vloženie (INSERT), vyhľadanie (SEARCH) alebo vymazanie (DELETE) určitého kľúča. Tieto operácie simulovali reálne používanie dátových štruktúr.

Ondrej Nebus ID:133751
Cvičenie Utorok: 16:00

Testovací proces

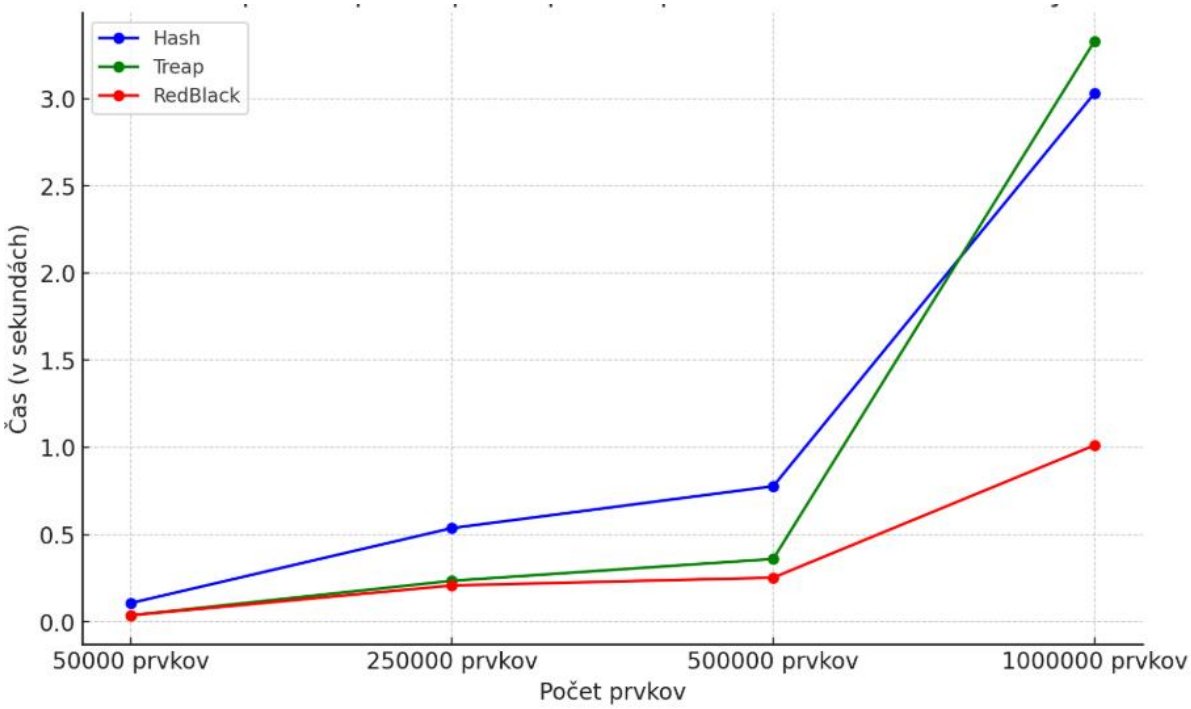
Testovanie prebiehalo nasledovne:

1. **Generovanie a načítanie testovacieho súboru**: V každom .c súbore (pre Hash, Treap, RedBlack) sa vytvorila funkcia, ktorá načítavala testovací súbor a vykonávala operácie podľa jednotlivých príkazov. Testovací súbor obsahoval operácie **INSERT** (na pridanie kľúča), **SEARCH** (na hľadanie kľúča) a **DELETE** (na vymazanie kľúča).
2. **Operácie vykonávané na dátových štruktúrach**:
    - **INSERT**: Vkladanie kľúča do dátovej štruktúry. Po každom vkladaní sa kontrolovalo, či sa kľúč správne uložil.
    - **SEARCH**: Vyhľadávanie kľúča v dátovej štruktúre. Výsledok bol buď "nájdený" alebo "nenájdený" v závislosti od toho, či sa kľúč nachádzal v štruktúre.
    - **DELETE**: Odstránenie kľúča zo štruktúry. Po vymazaní sa kontrolovalo, či bol kľúč úspešne odstránený.
3. **Meranie času**: Pre každý testovací súbor sa zaznamenal čas potrebný na vykonanie všetkých operácií. Použil som funkciu `clock()`, ktorá merala čas pred a po vykonaní všetkých operácií. Na konci sa vypočítal celkový čas v sekundách, ktorý bol potrebný na spracovanie všetkých operácií v danom testovacom súbore.

Výsledky:

Dátová štruktúra	50000 prvkov (s)	250000 prvkov (s)	500000 prvkov (s)	1000000 prvkov (s)
Hašovacia tabuľka (Hash)	0.1070	0.5370	0.7770	3.0300
Treap	0.0370	0.2350	0.3600	3.3290
Red-Black strom	0.0370	0.2080	0.2530	1.0120

Ondrej Nebus ID:133751
Cvičenie Utorok: 16:00



Záver

Pri testovaní a porovnávaní výkonu rôznych dátových štruktúr, konkrétne **Hašovacej tabuľky**, **Treap stromu** a **Red-Black stromu**, sa ukázalo, že **Red-Black strom** dosahuje najlepšie výsledky z hľadiska efektivity. Tento záver možno pripísať spôsobu, akým som testoval jednotlivé dátové štruktúry.

Na rozdiel od testovania jednotlivých operácií ako **INSERT**, **SEARCH** a **DELETE** samostatne, som počas testov spúšťal všetky tieto operácie v rámci jedného testu, čo sa ukázalo ako efektívnejší spôsob hodnotenia celkového výkonu dátovej štruktúry. Tento prístup simuluje reálny scenár, kde sa na strom aplikuje viacero operácií naraz, čo je častý prípad v reálnych aplikáciách.

Red-Black strom sa v týchto testoch správal najlepšie, pretože jeho schopnosť samovyvažovania počas vykonávania operácií umožnila rýchle a stabilné spracovanie aj pri vysokých objemoch dát. Naopak, pri **hašovacích tabuľkách** a **Treap stromoch** sa pri väčších objemoch dát občas objavili problémy s výkonom, čo môže byť spôsobené rôznymi faktormi, ako sú častejšie kolízie a potreba opakovaného **rehashovania**.

V prípade hašovacích tabuliek je dôležité zohľadniť aj fakt, že **pamäť v počítači** zohráva dôležitú úlohu. Pri väčšom počte prvkov sa totiž vyžaduje viac pamäte na uloženie položiek a na vykonanie **rehashovania**, čo môže viesť k spomaleniu výkonu, najmä ak pamäťová kapacita počítača nie je dostatočná. Tento faktor môže ovplyvniť výkon hašovacích tabuliek pri práci s veľmi veľkými dátovými súbormi.